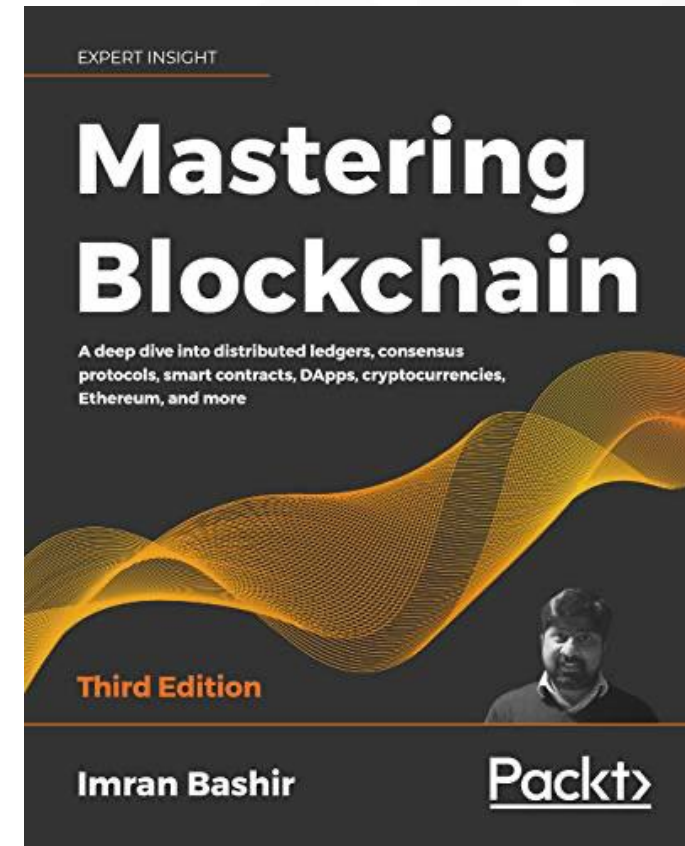


Mastering Blockchain

Third Edition

Chapter 18, Tokenization



Outline

- Tokenization on the blockchain
- Types of token
- Token offerings
- Token standards
- Building an ERC-20 token



What is tokenization?



- Tokenization converts an asset to a digital token on a blockchain.
- It is a process of converting the ownership rights of a real-world asset into a cryptographic/digital token on a blockchain.

Advantages of tokenization



- Flexibility
- Faster transaction processing
- Low cost
- Decentralization
- Security
- Transparency
- Trust
- Fractional ownership
- Low entry barrier
- Innovative applications
- More liquidity

Disadvantages of tokenization

- Regulatory issues
- Legality of tokens
- Technology barrier
- Security issues



Types of tokens



- **Fungible tokens**

- Indistinguishable from other tokens of the same type
- Interchangeable
- Divisible between multiple parties

- **Non-fungible tokens**

- Unique
- Non-interchangeable
- Indivisible

Stable tokens



- Stable token value is pegged to another asset's value.
- Types of stable token include:
 - Fiat collateralized
 - Commodity collateralized
 - Crypto collateralized
 - Algorithmically stable

Process of tokenization



- Onboard an investor.
- Asset is scrutinized and audited.
- Security Token Offering made.
- Physical asset is placed with a custodian
- Derivative token, representing the asset, released
- Trading on secondary markets begins

Token offerings



- Initial Coin Offerings (ICOs)
- Security Token Offerings (STOs)
- Initial Exchange Offerings (IEOs)
- Equity Token Offerings (ETOs)
- Decentralized autonomous Initial Coin Offerings (DAICOs)

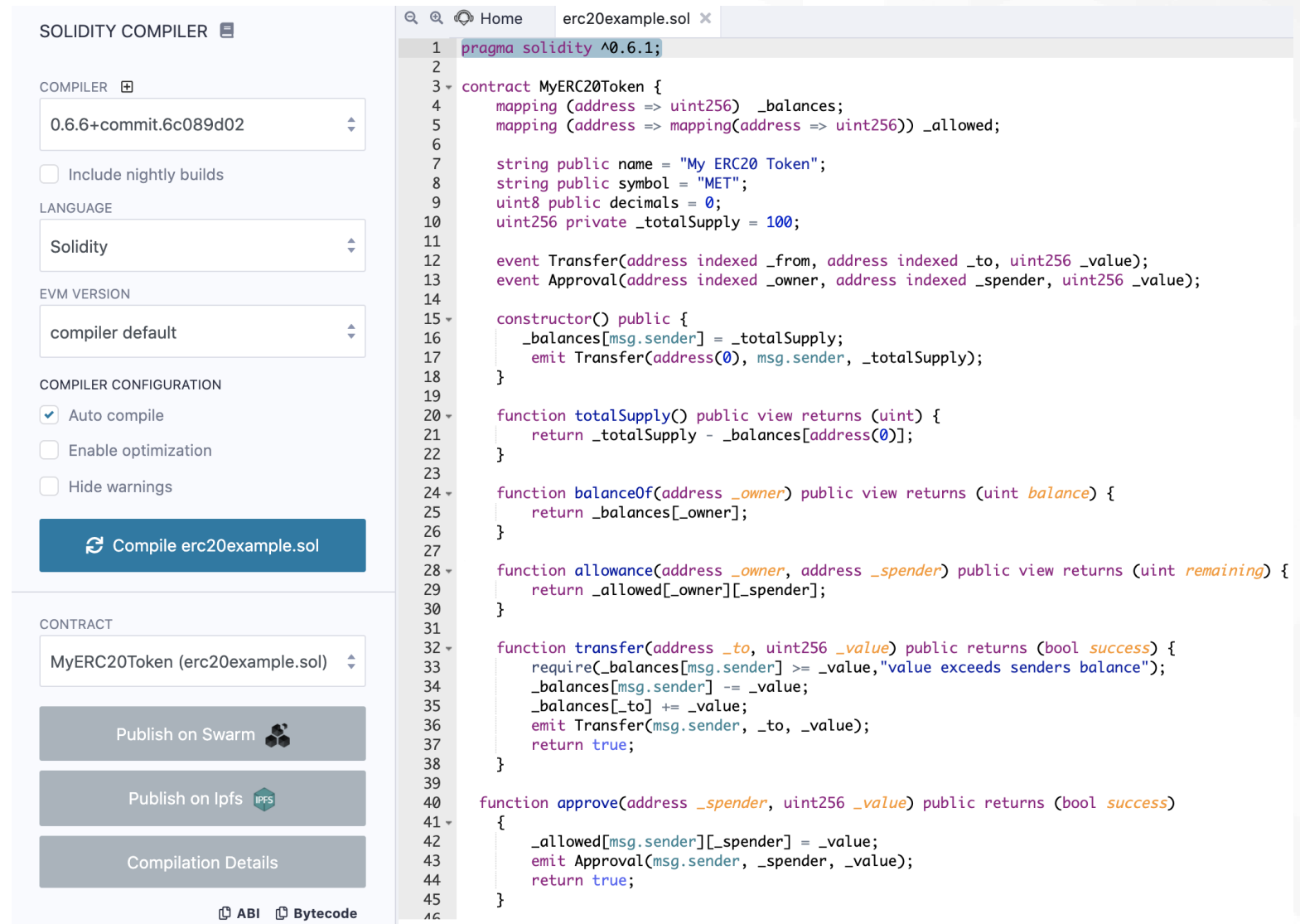
Token standards

- ERC-20
- ERC-223
- ERC-777
- ERC-721
- ERC-884
- ERC-1400
- ERC-1404

ERC-20

ERC-20 is the most famous token standard on the Ethereum platform.

Many token offerings are based on ERC-20 and there are wallets available, such as MetaMask, that support ERC-20 tokens.



The image shows the Remix IDE interface. On the left, the 'SOLIDITY COMPILER' panel is visible, showing the compiler version '0.6.6+commit.6c089d02', language 'Solidity', and EVM version 'compiler default'. The 'COMPILER CONFIGURATION' section has 'Auto compile' checked. A 'Compile erc20example.sol' button is present. Below, the 'CONTRACT' section shows 'MyERC20Token (erc20example.sol)' selected, with buttons for 'Publish on Swarm', 'Publish on Ipfs', and 'Compilation Details'. At the bottom, there are links for 'ABI' and 'Bytecode'.

The main editor displays the Solidity code for 'erc20example.sol'. The code defines a contract 'MyERC20Token' with the following features:

- Pragma: `pragma solidity ^0.6.1;`
- State variables: `mapping (address => uint256) _balances;` and `mapping (address => mapping(address => uint256)) _allowed;`
- Public variables: `string public name = "My ERC20 Token";`, `string public symbol = "MET";`, `uint8 public decimals = 0;`, and `uint256 private _totalSupply = 100;`
- Events: `event Transfer(address indexed _from, address indexed _to, uint256 _value);` and `event Approval(address indexed _owner, address indexed _spender, uint256 _value);`
- Constructor: `constructor() public { _balances[msg.sender] = _totalSupply; emit Transfer(address(0), msg.sender, _totalSupply); }`
- Function `totalSupply()`: `public view returns (uint) { return _totalSupply - _balances[address(0)]; }`
- Function `balanceOf(address _owner)`: `public view returns (uint balance) { return _balances[_owner]; }`
- Function `allowance(address _owner, address _spender)`: `public view returns (uint remaining) { return _allowed[_owner][_spender]; }`
- Function `transfer(address _to, uint256 _value)`: `public returns (bool success) { require(_balances[msg.sender] >= _value, "value exceeds senders balance"); _balances[msg.sender] -= _value; _balances[_to] += _value; emit Transfer(msg.sender, _to, _value); return true; }`
- Function `approve(address _spender, uint256 _value)`: `public returns (bool success) { _allowed[msg.sender][_spender] = _value; emit Approval(msg.sender, _spender, _value); return true; }`

ERC-20 token code in the Remix IDE

Exercise

Following the steps laid out in the *Mastering Blockchain* core book, complete the exercise *Building an ERC-20 token*.

Summary

In this presentation, we covered:

- Tokenization on the blockchain
- Types of token
- Token offerings
- Token standards
- Building an ERC-20 token